

Agile

гибкие методологии разработки

История

+ 1950: ВОЗНИКНОВЕНИЕ

- сверх талантливые люди решали проблемы в несложных программных средах

+ 1960: ВЫХОД В СВЕТ

- университеты - "компьютерные курсы"
- отсутствие персональных средств компиляции

+ 1970: ДЕСЯТИЛЕТИЕ ХАОСА

- появление ПК

История

+ 1980: ВОЗРОЖДЕНИЕ

- вспомогательные средства программирования
- формальные методы

+ 1990: СОВЕРШЕНСТВОВАНИЕ

- совершенствование процесса разработки программ
- 1995 - Scrum
- 1996 - XP, Lean, Kanban

+ 2000: РАЗВИТИЕ

- гибкие методологии разработки
- 2001 - agile-манифест

История

+ 2010: КООПЕРАЦИЯ

- DevOps - усиление кооперации между группами разработки (DEVelopments) и эксплуатации (OPerations)

Каскадная модель

(Традиционная, или модель Водопад, Waterfall)

- 1970 - Уинстон Ройс
"Управление разработкой крупных программных систем"
- 70е - 80е - стандарт министерства обороны США

Каскадная модель

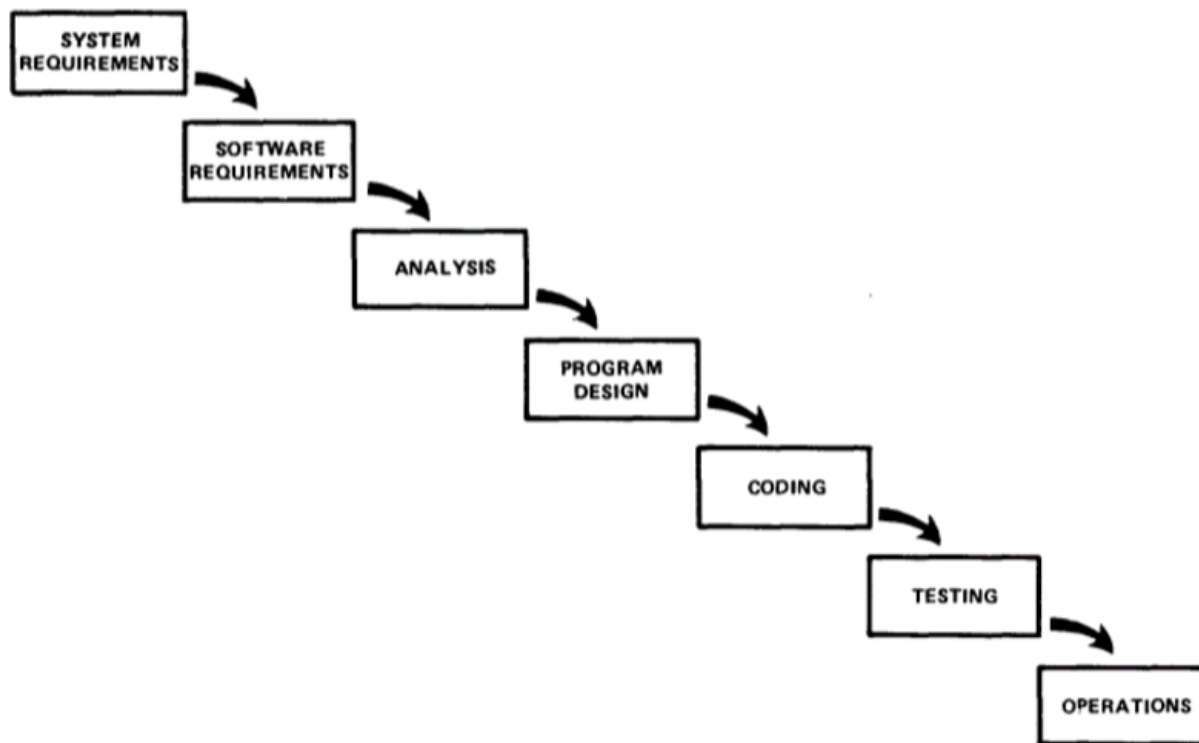


Figure 2. Implementation steps to develop a large computer program for delivery to a customer.

Каскадная модель

Минусы

- Возвращение на предыдущий этап не предусмотрено
- Чем больше проект тем больше накапливается проблем в процессе разработки
- При появлении критических ошибок весь проект начинается сначала

Каскадная модель

Плюсы

- Строгие требования и высокая степень формализации
- Дисциплинированность
~~"пишем и правим"~~
- 1. Понимание целей
2. Реализация продукта

Спиральная модель

(Spiral model)

- 1988 - Барри Бозм
"Спиральная модель разработки и улучшения ПО"

Десять наиболее распространённых (по приоритетам) рисков:

- Дефицит специалистов.
- Нереалистичные сроки и бюджет.
- Реализация несоответствующей функциональности.
- Разработка неправильного пользовательского интерфейса.
- «Золотая сервировка», перфекционизм, ненужная оптимизация и оттачивание деталей.
- Непрерывающийся поток изменений.
- Нехватка информации о внешних компонентах, определяющих окружение системы или вовлечённых в интеграцию.
- Недостатки в работах, выполняемых внешними (по отношению к проекту) ресурсами.
- Недостаточная производительность получаемой системы.
- «Разрыв» в квалификации специалистов разных областей знаний.

Спиральная модель

Каждый виток разбит на 4 сектора:

- оценка и разрешение рисков,
- определение целей,
- разработка и тестирование,
- планирование.

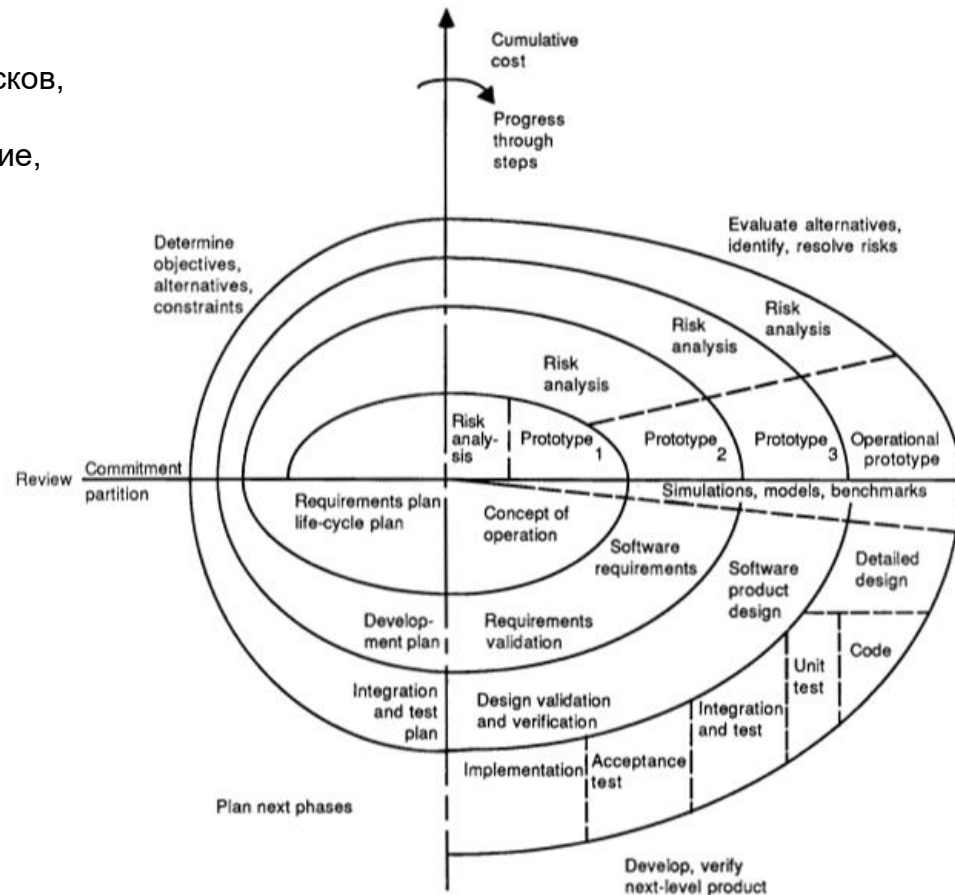


Figure 2. Spiral model of the software process.

Спиральная модель

- Каждая фаза водопадного процесса разработки в спиральной методологии завершается этапом прототипирования и управления рисками.
- Этап прототипирования после каждой фазы проекта позволяет определить, насколько текущее состояние проекта соответствует первоначальному плану.
- По итогам прототипирования выполняется либо переход к следующей фазе, либо возвращение на одну из предыдущих фаз.

Спиральная модель

Плюсы

- Стабильность архитектуры
- Большие, дорогостоящие и сложные проекты

Спиральная модель

Минусы

- Определение момента перехода на следующий этап

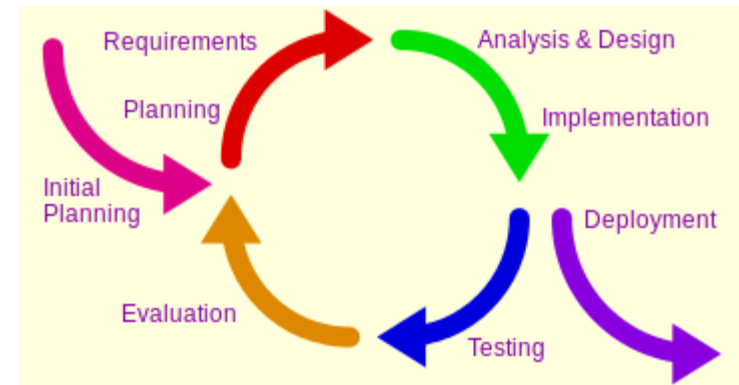
Итеративная модель

(Iterative and incremental development, IID)

- Истоки — 1930е года
- Принцип разделения на итерации
- Каждая итерация — мини-проект
- Важная веха — проект по разработке самолета X-15

Итеративная модель

- Преимущества итеративного подхода:
- снижение воздействия серьёзных [рисков](#) на ранних стадиях проекта, что ведет к минимизации затрат на их устранение;
- организация эффективной обратной связи проектной команды с потребителем (а также заказчиками, [стейкхолдерами](#)) и создание продукта, реально отвечающего его потребностям;
- акцент усилий на наиболее важные и критичные направления проекта;
- непрерывное итеративное тестирование, позволяющее оценить успешность всего проекта в целом;
- раннее обнаружение конфликтов между требованиями, моделями и реализацией проекта;
- более равномерная загрузка участников проекта;
- эффективное использование накопленного [опыта](#);
- реальная оценка текущего состояния проекта и, как следствие, большая уверенность заказчиков и непосредственных участников в его успешном завершении.
- затраты распределяются по всему проекту, а не группируются в его конце[\[1\]](#).



Пример реализации итеративного подхода — [Rational Unified Process](#).

Итеративная модель

Плюсы

- Ранняя обратная связь
- Возможность "отката"

Итеративная модель

Минусы

- Отсутствие долгое время целостного понимания возможностей и ограничений проекта
- Часть работы неизбежно приходится отбрасывать
- Снижение добросовестности и дисциплины

Гибкие методологии

- Принцип разделения на итерации
- Человеческий фактор
- Конечный продукт
- Ограничения по времени

Agile-манифест

Люди и
взаимодействие

важнее

процессов и
инструментов

Работающий
продукт

важнее

исчерпывающей
документации

Сотрудничество с
заказчиком

важнее

согласования условий
контракта

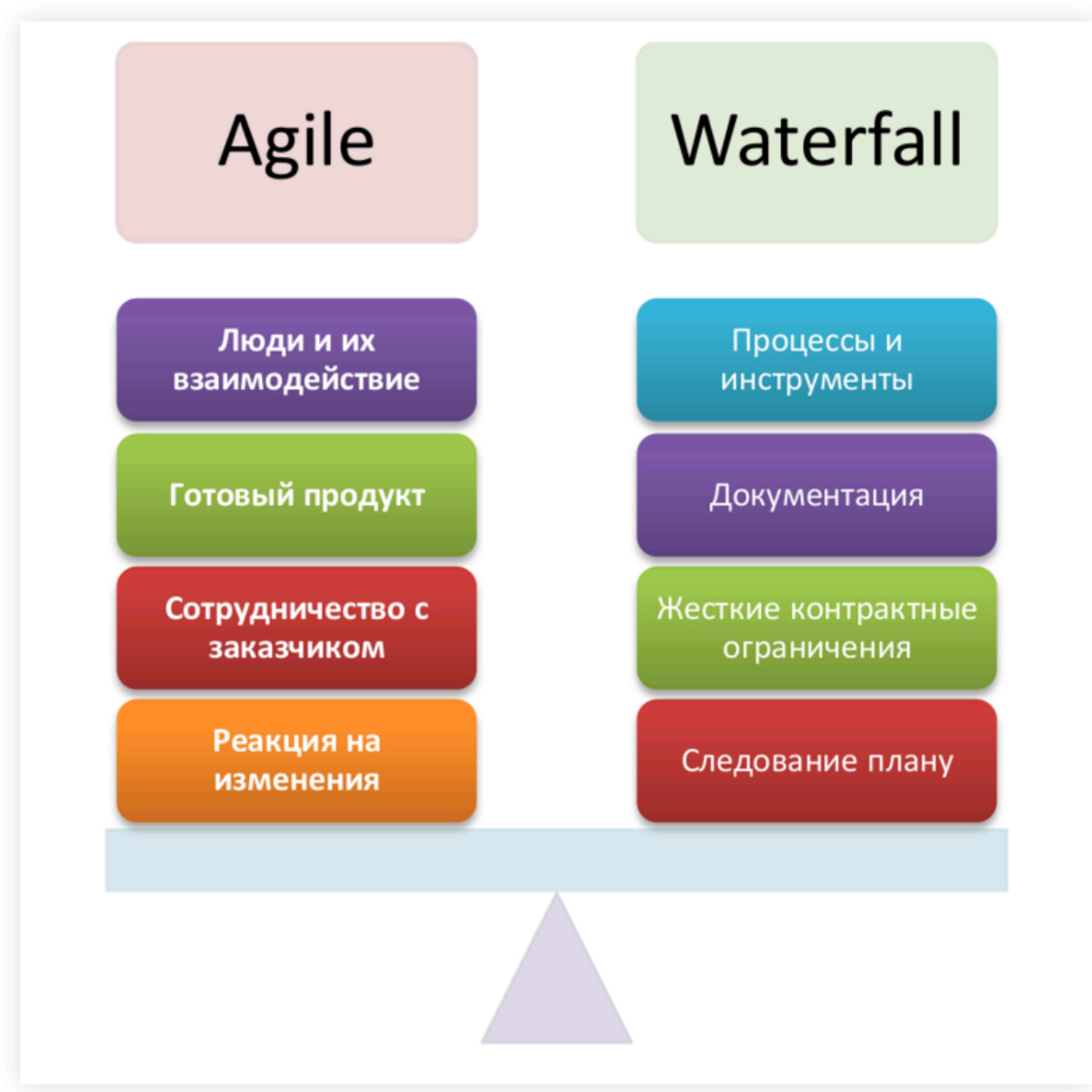
Готовность к
изменениям

важнее

следования
первоначальному
плану

agilemanifesto.org

Визуализация ценностей манифеста гибкой разработки



Гибкие методологии

Термин Agile

- Мартин Фаулер - сомнения по произношению

Словари ABBYY Lingvo (En-Ru)

agile

[**'ædʒaɪl**] *брит.*  / *амер.* 

прил.

- 1) проворный; быстрый, живой, подвижной, расторопный, шустрый
- 2) сообразительный

Гибкие методологии



Экстремальное программирование (Extreme Programming, XP)

Управленческие практики

- Игра в планирование
- Частые небольшие релизы
- Заказчик всегда рядом
- 40-часовая рабочая неделя
- Коллективное владение кодом

Инженерные практики

- Непрерывная интеграция
- Парное программирование
- Разработка через тестирование
- Рефакторинг
- Простота
- Метафора системы
- Стандарт кодирования

Канбан (Kanban, 看板)



- Визуализируйте производственный процесс
- Ограничивайте количество незавершенной работы
- Оптимизируйте процесс

Скрам (Scrum)

- Скрам (Scrum) — это набор принципов, на которых строится процесс разработки, позволяющий в жёстко фиксированные и небольшие по времени итерации, называемые спринтами (sprints), предоставлять конечному пользователю работающее ПО с новыми возможностями, для которых определён наибольший приоритет.
- Возможности ПО к реализации в очередном спринте определяются в начале спринта на этапе планирования и не могут изменяться на всём его протяжении.
- При этом строго фиксированная небольшая длительность спринта придаёт процессу разработки предсказуемость и гибкость.

Скрам

- Хиротака Такэути и Икудзиро Нонака
статья The New Product Development Game
- Небольшие команды из специалистов различного
профиля
- Scrum - толкотня, схватка вокруг мяча в регби

Скрам: драка за мяч в регби

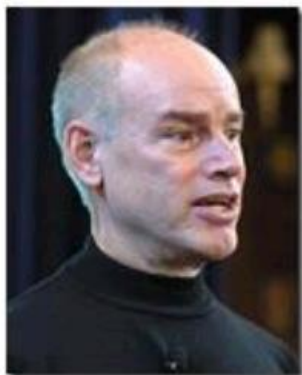


Регби, Кубок Европейских наций, России - Грузия, 23 февраля 2013

Скрам: основатели



Jeff Sutherland



Ken Schwaber

Джеф Сазерленд и Кен Швабер

Скрам: выход в свет

- 1995 - Объектно-ориентированное программирование, системы, языки и приложения. (OOPSLA '95) Остин, штат Техас
- 2001 - «Гибкая разработка ПО со Скрам», Кен Швабер и Майк Бидл
- Официальное «Руководство по Скраму» - scrum.org

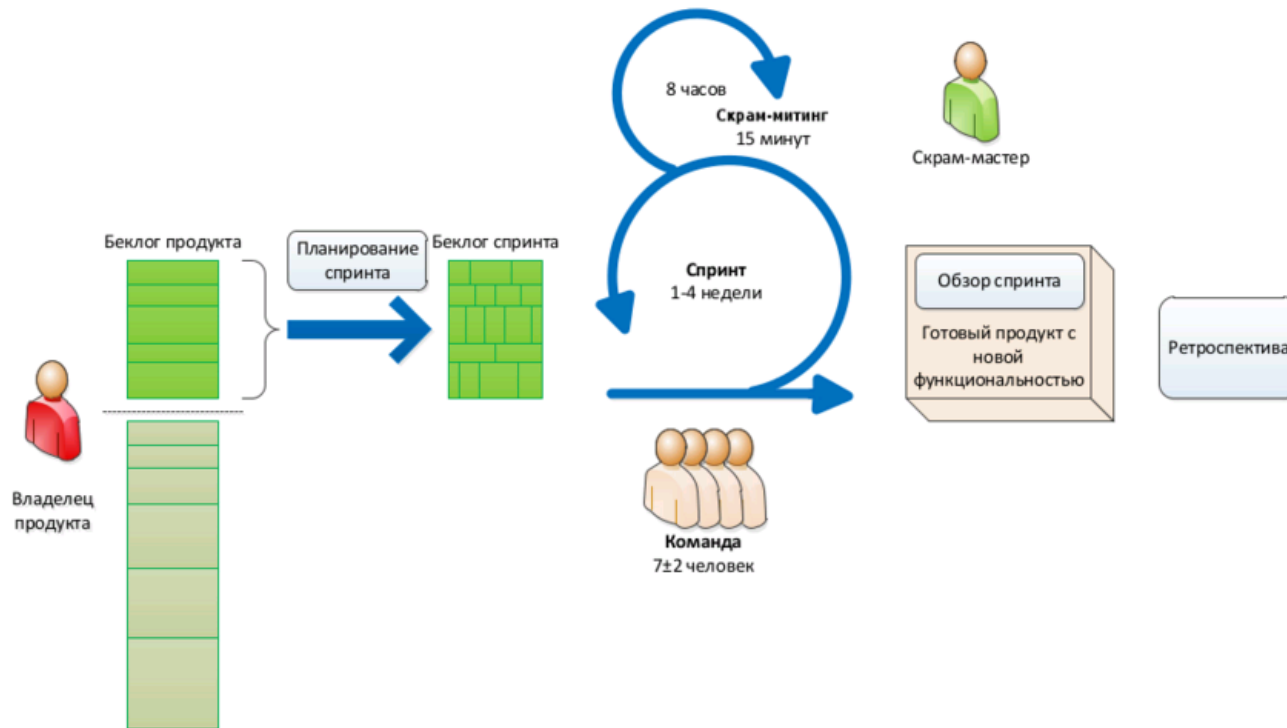
Скрам: одной фразой

Скрам - это подход для разработки и поддержки функционально сложных продуктов

Скрам: ещё одной фразой

Скрам - это фреймворк для управления творческими командами для создания продукта наивысшего качества

Скрам: одной схемой



Элементы скрам

Роли

- Владелец продукта
- Скрам-мастер
- Команда

Артефакты

- Беклог продукта
- Беклог спринта
- Инкремент продукта

Процессы

- Планирование спринта
- Обзор спринта
- Ретроспектива
- Скрам-митинг
- Спринт

Скрам Команда

- Владелец Продукта (Product Owner)
человек, отвечающий за комплектацию спринтов и за приоретизацию задач
- Команда Разработчиков (Development Team)
люди, непосредственно реализующие инкремент спринта
- Скрам Мастер (Scrum Master)
человек в команде, отвечающий за Скрам

Артефакты

- Журнал Продукта (Product Backlog)
список требований, по выполнению которых
разработка продукта будет считаться завершенной
- Журнал Спринта (Sprint Backlog)
часть журнала продукта отобранная для спринта
- Инкремент продукта (Increment)
новая функциональность продукта, созданная во
время спринта

Артефакты описание

- **Бэклог проекта** — это список требований к функциональности, упорядоченный по их степени важности, подлежащих реализации. Элементы этого списка называются «пожеланиями пользователя» (*user story*) или элементами бэклога (*backlog items*). **Бэклог проекта** открыт для редактирования для всех участников скрам процесса.
- **Бэклог спринта** — содержит функциональность, выбранную **владельцем проекта** из **Бэклога проекта**. Все функции разбиты по задачам, каждая из которых оценивается **скрам-командой**. Каждый день команда оценивает объём работы, который нужно проделать для завершения спринта[\[11\]](#).
- **Задачи истории спринта (Sprint Story Tasks)** Добавляются к историям спринта. Выполнение каждой задачи оценивается в часах. Каждая задача не должна превышать 12 часов (зачастую команда настаивает, чтобы максимальная продолжительность задачи равнялась одному рабочему дню).

Дополнительные определения

- **История спринта (Sprint Story)**
- Требуемую функциональность, которую добавляют в бэклог, часто называют историей. Зачастую история имеет следующую структуру: «Будучи пользователем <тип пользователя> я хочу сделать <действие>, чтобы получить <результат>». Такая структура удобна тем, что понятна как разработчикам так и заказчикам.
- **Остановка спринта (Abnormal Termination)**
- Остановка спринта может быть произведена раньше срока его планового окончания в исключительных ситуациях. Спринт может остановить команда, если понимает, что не может достичь цели спринта в отведённое время. Спринт может остановить **владелец проекта**, если исчезает необходимость в реализации цели спринта. После остановки спринта проводится совещание с командой, где обсуждаются причины остановки. После этого начинается новый спринт.
- **Очки за пользовательскую историю (Story Points)**
- Абстрактная метрика оценки сложности истории, которая не учитывает затраты в человекочасах. Обычно используют одну из следующих шкал: [ряд Фибоначчи](#) (1,2,3,5,8,13,21,34,55); линейную шкалу (1,2,3,4 ... n); степень двойки (1,2,4,8 ... 2n); размеры одежды (XS, S, M, L, XL).
- **Критерий готовности (Definition of Done (DoD))**
- Критерии, определяющие степень готовности элемента из *журнала пожеланий пользователя*.
- **Скорость команды (Velocity)**
- Общее количество очков, набранных командой за предыдущий спринт. Данная метрика помогает команде понять, сколько историй она может сделать за один спринт.

Мероприятия

- Спринт (Sprint)
одна полная итерация процесса разработки
- Планирование Спринта (Sprint Planning)
отбор задач на спринт
- Ежедневные Скрамы (Daily Scrum)
ежедневная планерка для всей команды
- Обзор Спринта (Sprint Review)
встреча в конце спринта для показа владельцу функционала продукта, сделанного за спринт
- Ретроспектива Спринта (Sprint Retrospective)
обсуждение результатов спринта

Скрам Команда

Владелец Продукта

- Один человек
- Определяет требования к продукту, знает, каким он должен быть в итоге
- Определяет дату релиза
- Корректирует приоритеты на каждой итерации
- Принимает работу

Владелец Продукта

- Четкое определение элементов Журнала Продукта
- Упорядочение элементов Журнала Продукта для оптимизации достижения целей и поставленных задач
- Обеспечение доступности, прозрачности и понятности Журнала Продукта
- Ответственность за понимание Командой Разработчиков требований Журнала Продукта на надлежащем уровне

Команда Разработчиков

- Самоуправляемы
- Кроссфункциональны
- Одна должность - Разработчик
- Ответственность лежит на всей Команде Разработчиков
- У Команды Разработчиков нет структурных подразделений

Команда Разработчиков

Размер

простота в
управлении

vs

выполнение
большого объёма
работы

от 5 до 9 человек

Скрам Мастер

- Понимание Скрама всеми участниками
- Понимает и практикует гибкие методы разработки и управления
- При необходимости проводит мероприятия Скрама
 - Планирование и запуск Спринта
 - Обзор и ретроспектива Спринта
- Мониторинг социальных аспектов команды и поддержание командного духа

Мероприятия

Спринт

- Скрам проекты развиваются сериями "спринтов"
- Типичная продолжительность – от 2 до 4 недель с жестким ограничением по времени
- Постоянная продолжительность спринта привносит ритм в разработку
- Продукт проектируется, разрабатывается и тестируется на протяжении одного спринта

Характеристики спринта

- Спринт [\[10\]](#) — итерация в скраме, в ходе которой создаётся функциональный рост программного обеспечения.
- Жёстко фиксирован по времени. Длительность одного спринта от 2 до 4 недель. В отдельных случаях, к примеру согласно Scrum стандарту Nokia, длительность спринта должна быть не более 6 недель.
- Тем не менее, считается, что чем короче спринт, тем более гибким является процесс разработки, релизы выходят чаще, быстрее поступают отзывы от потребителя, меньше времени тратится на работу в неправильном направлении.
- С другой стороны, при более длительных спринтах команда имеет больше времени на решение возникших в процессе проблем, а владелец проекта уменьшает издержки на совещания, демонстрации продукта и т. п.
- Разные команды подбирают длину спринта согласно специфике своей работы, составу команд и требований, часто методом проб и ошибок.
- Для оценки объёма работ в спринте можно использовать предварительную оценку, измеряемую в очках истории.
- Предварительная оценка фиксируется в бэклоге проекта.
- На протяжении спринта никто не имеет права менять список требований к работе, внесённых в бэклог спринта.

Спринт: параллельная разработка

Требования

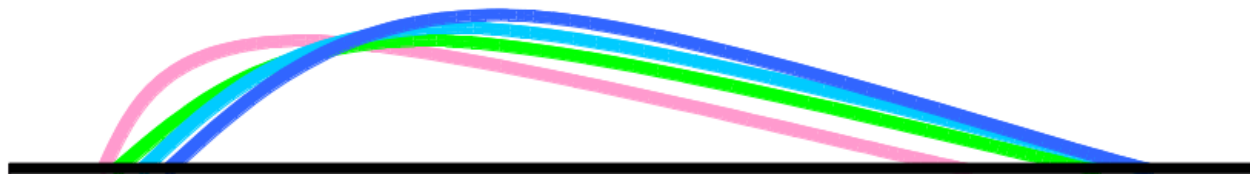
Дизайн

Разработка

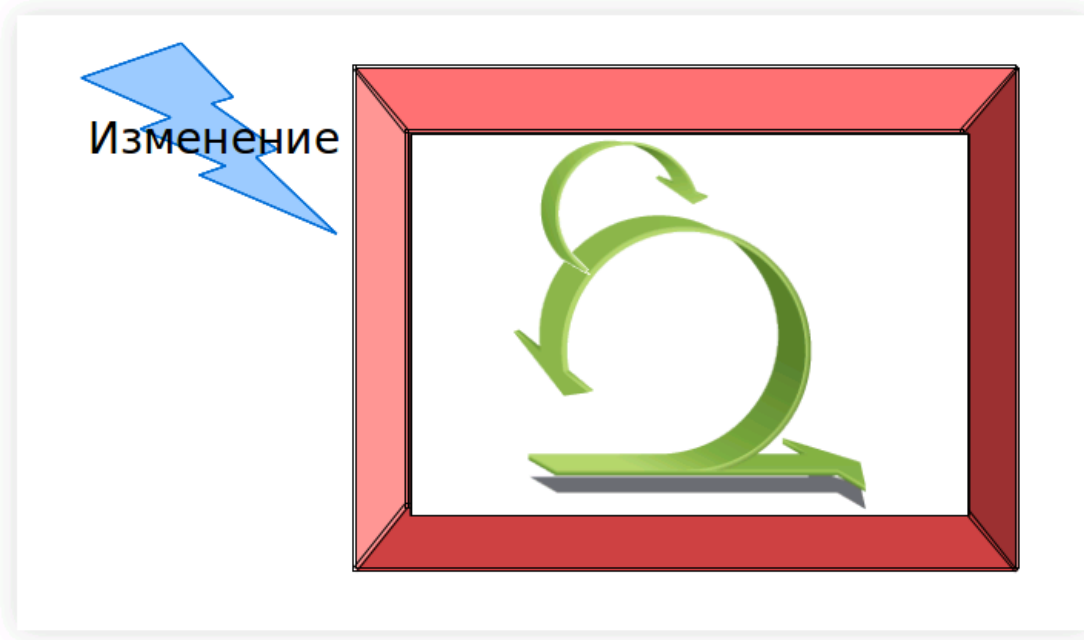
Тестирование

Вместо того, чтобы
выполнять эти активности
по очереди...

... Скрам команды делают
понемногу от каждой все
время



Никаких изменений в течении спринта



- Планируйте длительность спринта исходя из соображения о том, как долго вы можете работать, не внося изменения в план работ

Планирование спринта

- Команда выбирает из Бэклога Продукта требования, которые они могут реализовать за спринт
- Создается Бэклог Спринта
- Задачи идентифицируются и оцениваются (1-16 часов)
- Все делается командой, а не Скрам Мастером
- Учитывается высокоуровневая архитектура приложения

Ежедневный скрам

- Характеристики
 - Ежедневно
 - 15 минут
 - Стоя
- Не для решения проблем
 - Приглашены все желающие
 - Только участники команды могут говорить (владелец продукта - тоже часть команды)
- Скрам Мастер лишь ведет собрание



Каждый отвечает на три вопроса

1

Что ты сделал вчера?

2

Что будешь делать сегодня?

3

Что тебе мешает?

- Это **HF** статусный отчет Scrum Master

Обзор Спринта

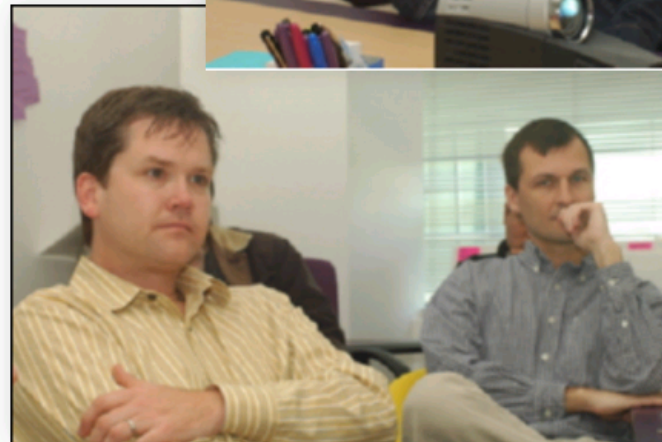
- Команда представляет, что было сделано за спринт
- Фокус на результат, а не процесс
- Обычно принимает форму демонстрации
- Неформально
 - Максимум 2 часа на подготовку
 - Без слайдов
- Вся команда участвует
- Приглашены все, кому может быть интересно

Обзор Спринта: обратная связь



Ретроспектива

- Периодический пересмотр того, что работает, а что нет
- Обычно 15-30 минут
- После каждого спринта
- Участвует вся команда
- Можно пригласить
Владельца Продукта



Структура ретроспективы



Каждый отвечает на три вопроса

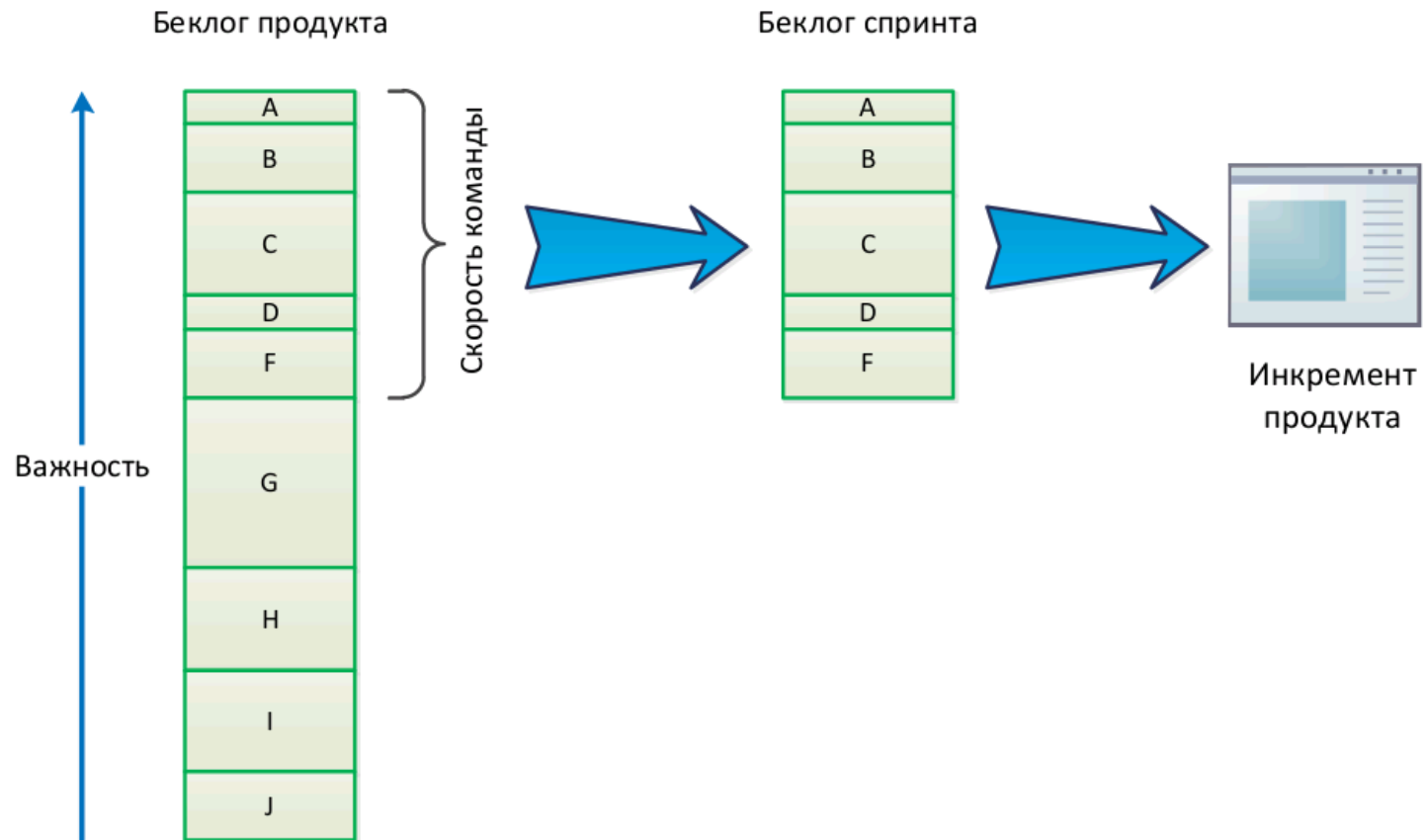
1. Что было сделано хорошо?
2. Что можно улучшить?
3. Какие улучшения будем делать?

- Основное правило

«В независимости от того, что удастся выяснить в результате ретроспективы, каждый член команды сделал всё, чтобы добиться успеха»

Артефакты

Артефакты



Журнал продукта

- Список требований оформляются в виде **историй пользователя**
 - Уникальный числовой идентификатор истории
 - Название истории пользователя
 - Важность
 - Оценка (**сторипоинты**)

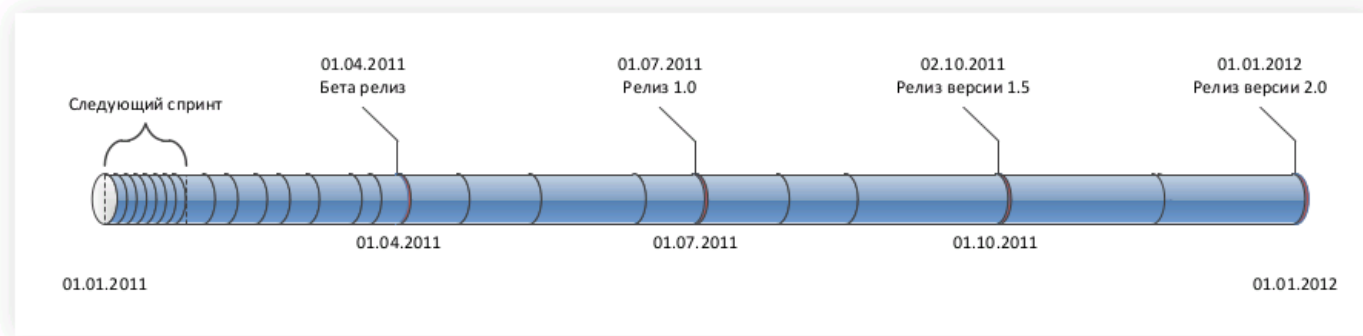
1453	
<i>Неавторизованный посетитель вводит логин и пароль для того, чтобы авторизоваться на сайте</i>	
200	10

Журнал продукта: планирование



- Конфликт при выборе размера Журнала Продукта

Метод накатывающей волны «rolling wave planning»



- Большие куски функциональности, в дальнейшем будут разбиты на маленькие
- Называются «эпиками» («epic»)

Как определить какое
количество сторипоинтов
назначить истории
пользователя?

Покер-планирование



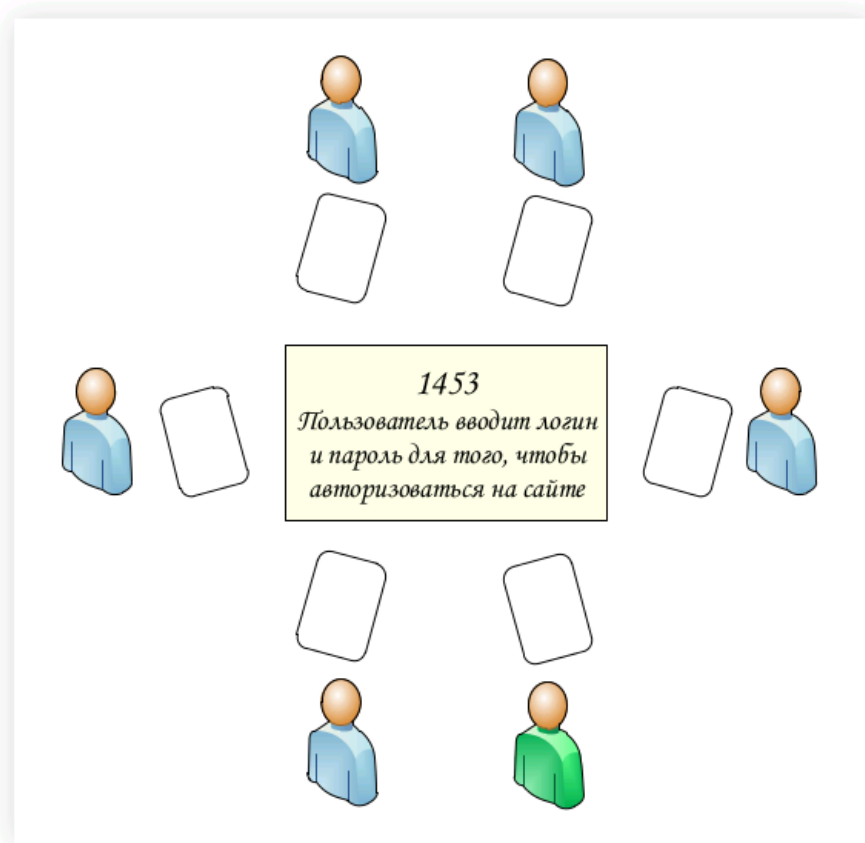
- Покер-планирование (Planning poker) – консенсусная относительная оценка историй пользователей командой

Покер-планирование

- Выбор эталонной задачи в 1 сторипоинт
 - простая и понятная для команды
 - типичная для данного проекта
 - небольшого размера
- Для оценки используется дискретная логарифмическая шкала

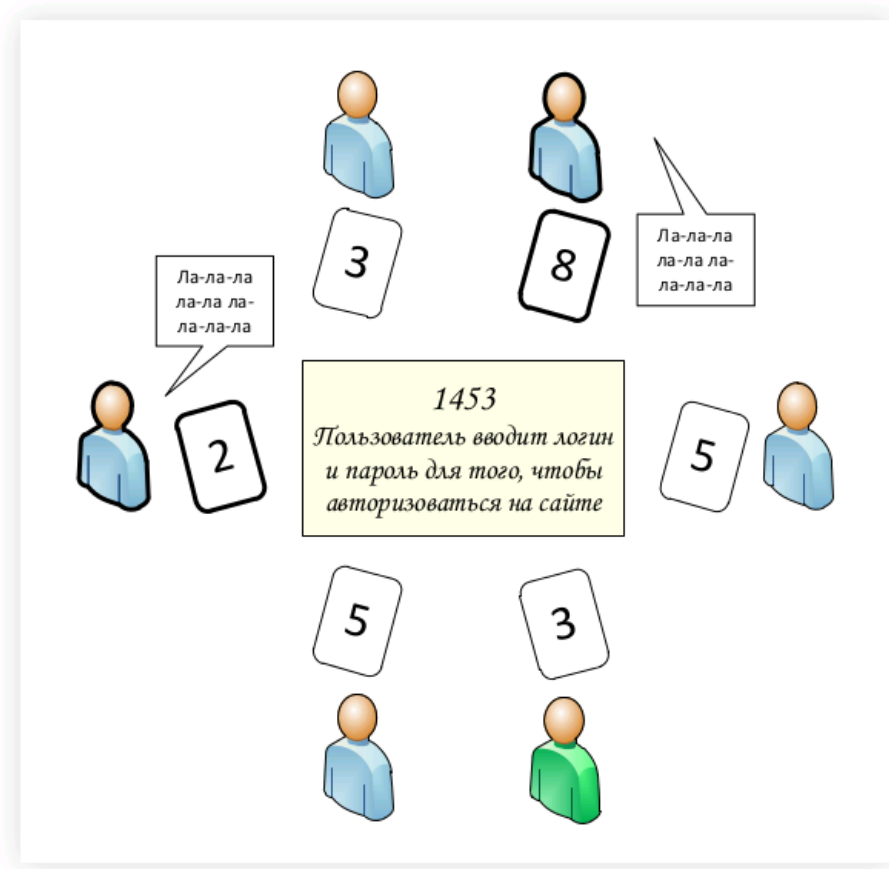
0	0,5	1	2	3	5	8	13	20	40	100
---	-----	---	---	---	---	---	----	----	----	-----

Покер-планирование: раунд 1



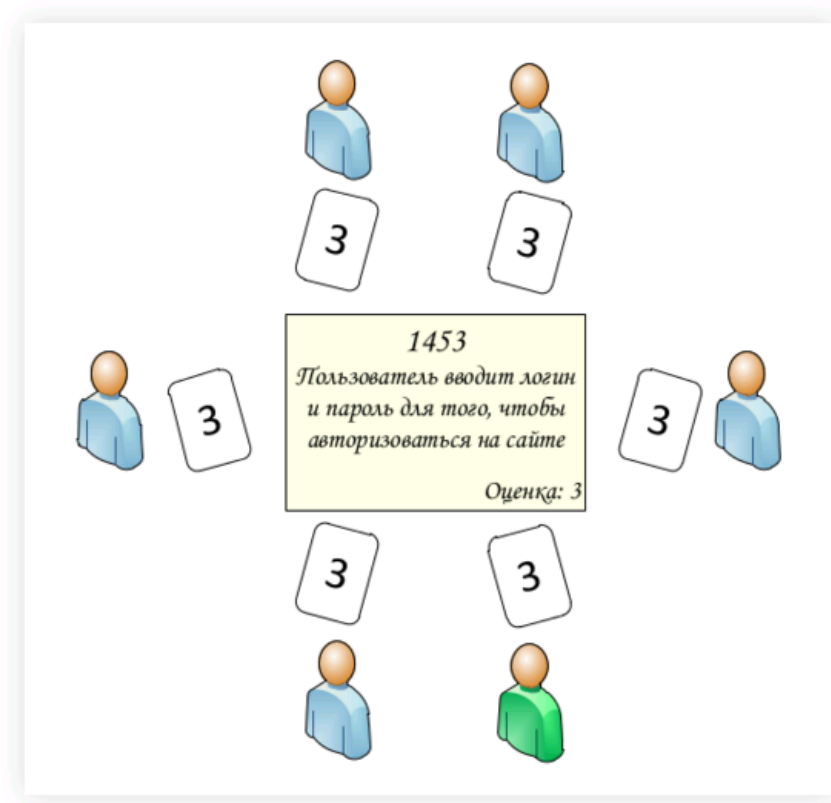
- Важно:
 - понимание сути истории каждым участником
 - отсутствие давления

Покер-планирование: раунд N



- Начинают обсуждение обычно с крайних оценок
- Срам Мастер следит за временем

Покер-планирование: консенсус



- Сглаживаются разногласия
- Владелец продукта начинает понимать процесс работы над задачей
- Команда точнее и детальнее понимает суть задачи

Покер-планирование: расклад

0	0,5	1	2	3	5	8	13	20	40	100
										
										
										
										
										
										
										
										
										
										
										

Подготовка покера

- Для проведения покера планирования необходимо подготовить список обсуждаемых функций и несколько колод пронумерованных карт. Список функций либо [пользовательские истории](#) описывают разрабатываемое программное обеспечение. Карты в колодах должны быть пронумерованы. Обычно колода содержит карты, содержащие [числа Фибоначчи](#), включая ноль: 0, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89; другие разновидности колод могут использовать аналогичные последовательности.
- Аргумент в пользу использования последовательности Фибоначчи — отражение типичной [неопределённости](#) при обсуждении самых важных и больших пунктов.
- Одна из имеющихся в продаже колод содержит следующую последовательность: 0, $\frac{1}{2}$, 1, 2, 3, 5, 8, 13, 20, 40, 100 и иногда *знак вопроса* («?»), означающий неуверенность, и *чашку кофе*, означающую требование перерыва.
 - Некоторыми организациями используются обычные игровые карты, включающие [туз](#), 2, 3, 5, 8 и [короля](#). Король буквально означает: «Данный пункт слишком большой или его слишком сложно оценить». Выбрасывание короля завершает обсуждение пункта в текущем круге ([англ. sprint](#)).
- По желанию, может использоваться [таймер](#), чтобы устанавливать лимит времени одного круга.

Процедура проведения

- Каждому участнику обсуждения выдаётся по одной колоде карт. Все колоды идентичны друг другу.
- Обсуждение проводится следующим образом.
- Ведущий ([англ. Moderator](#)), не участвующий в обсуждении, ведёт собрание.
- Менеджер проекта ([англ. Product Manager](#)) представляет краткие обзоры каждого из пунктов. Команда может задавать вопросы и вести обсуждение предложений и рисков. Итог обсуждения записывается менеджером проекта.
- Участники выбирают по одной карте и кладут их рубашкой вверх, показывая таким образом, что выбор сделан. Числовые достоинства карт могут использоваться по-разному: они могут означать количество дней, наиболее подходящие дни или относительные единицы сложности ([англ. story points](#)). Во время обсуждения достоинствам не должны приписываться новые значения в зависимости от размера функций с целью избегания [эффекта привязки](#).
- Каждый участник называет свою карту и переворачивает её.
- Участникам с высокими и низкими оценками даётся возможность высказаться и обосновать свою оценку.
- Процесс обсуждения продолжается до тех пор, пока не будет достигнут консенсус. Голос участника, который, скорее всего, будет владеть разработкой, имеет больший вес в «голосовании на основе консенсуса».
- Таймер используется для обеспечения структурированности обсуждения; ведущий или менеджер проекта может в любое время перезапустить таймер, по истечении времени все обсуждения должны быть прекращены, затем начинается новый круг покера.
- Выступления участников повторяются вновь и вновь. Карты пронумерованы так, что чем больше число, тем больше неопределённость. Так, если разработчик желает выбрать 6, но он не до конца уверен, он выберет 5, либо может предусмотрительно выбрать 8.

Достоинства метода

- Покер планирования — это средство оценки проектов по разработке программного обеспечения. Эта техника минимизирует эффект привязки путём опроса каждого из участников команды таким образом, что никто не знает чужого решения до одновременного оглашения выбора каждого из участников.
- Исследование [\[3\]](#) К. Молёккен-Эствольда ([норв.](#) *K. Moløkken-Østvold*) и Н. Хаугена ([норв.](#) *N.C. Haugen*) показало, что оценки, полученные с помощью покера планирования, были менее оптимистичными и более точными, чем оценки, полученные с помощью простого сложения отдельных оценок аналогичных задач.

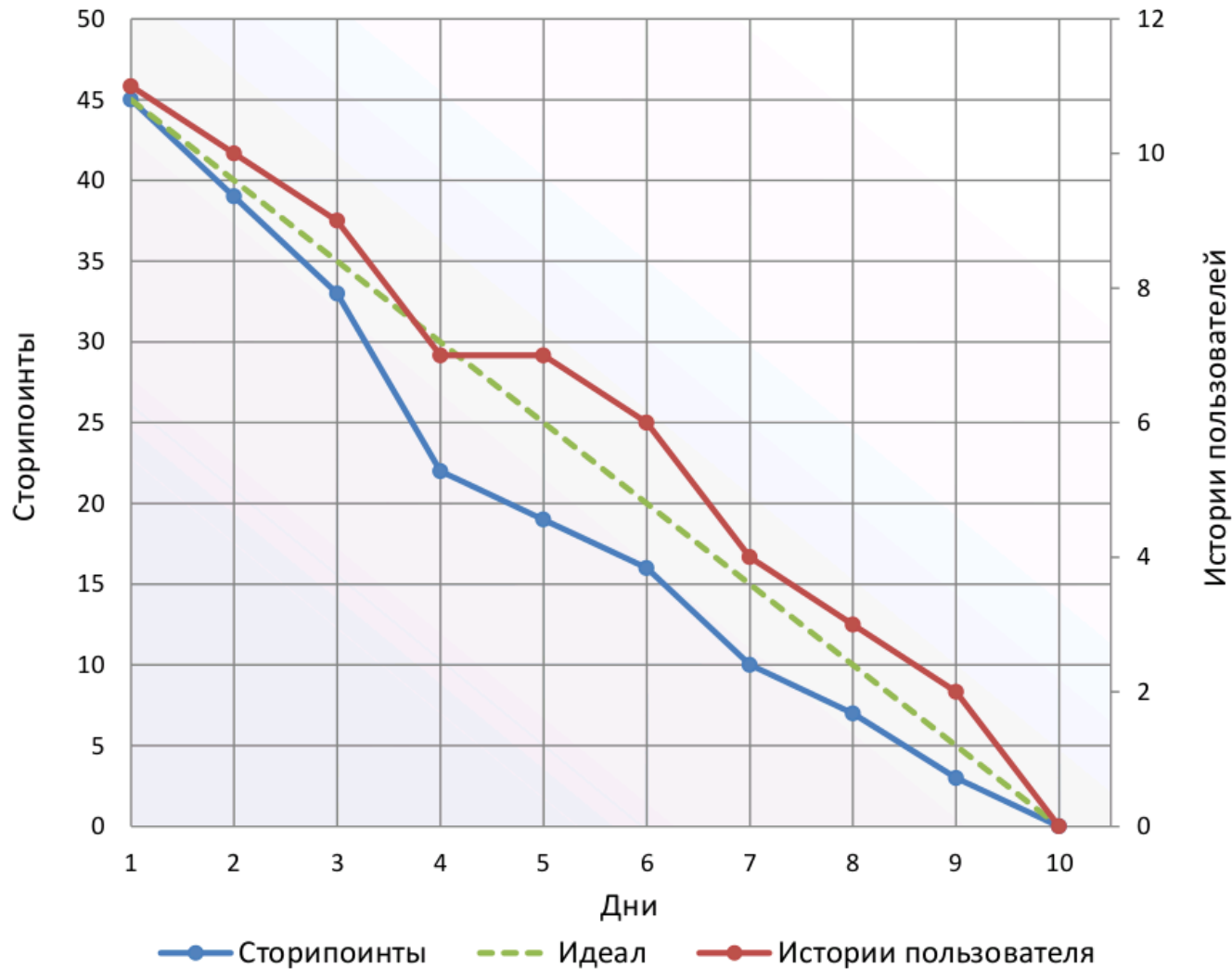
Избегание эффекта привязки

- Эффект привязки возникает, когда команда открыто обсуждает оценки. Команда обычно имеет в своём составе как сдержанных, так и импульсивных участников, могут быть участники, у которых есть определённые планы; разработчики, вероятно, захотят как можно больше времени заниматься работой над проектом, а владелец продукта или заказчик, вероятно, захочет, чтобы работа была закончена как можно скорее.
- Оценка становится подверженной эффекту привязки, когда владелец продукта говорит нечто подобное: «Я думаю, это несложная работа, вряд ли это займёт больше пары недель». Либо когда разработчик говорит: «Думаю, нам нужно лучше стараться; решение проблем с бэк-эндом, которые у нас были, могло затянуться на месяцы». Если начинающий обсуждение говорит: «Думаю, это займёт 50 дней», — он сразу устанавливает рамки мышления остальных участников; возникает эффект привязки, то есть число 50 подсознательно будет отправной точкой для всех участников.
- Те, кто хотел назвать число 100, захотят уменьшить свою оценку, а те, кто задумал число 10, захотят увеличить её. Это становится серьёзной проблемой, если число 50 произносится влиятельным участником в то время, когда остальная команда преимущественно останавливает свой выбор на больших или меньших значениях. Из-за эффекта привязки у остальных участников они могут — сознательно или нет — не проявить своего первоначального единомыслия; на самом деле, они могут так и не понять, что они думали об одном и том же. Это может быть опасным и приводить к оценкам, на которые повлияли планы или личные мнения людей, не сфокусированных на качественном выполнении задачи.
- Покер планирования выявляет потенциально влиятельного участника команды, изолируя его мнение от других участников группы. Затем необходимо, чтобы участник аргументировал свой выбор, если он не совпадает с преобладающим мнением. Если участники группы могут выражать свою сплочённость таким образом, они более склонны верить в свои первоначальные оценки.
- Если у влиятельного участника есть хорошие аргументы для спора, все остальные будут видеть смысл и прислушиваться, но, по крайней мере, остальные участники не будут подвержены эффекту привязки; вместо этого они должны будут исходить только из разумных соображений.

Как осуществляется
мониторинг прогресса?

Диаграмма сгорания

(Sprint burndown chart)



Доска задач



План	Аналитика	Разработка	Тестирование	Готово
A				
B				
C				
D				
E				

- все истории находятся в первом столбце
- отсортированы вертикально по важности

Доска задач



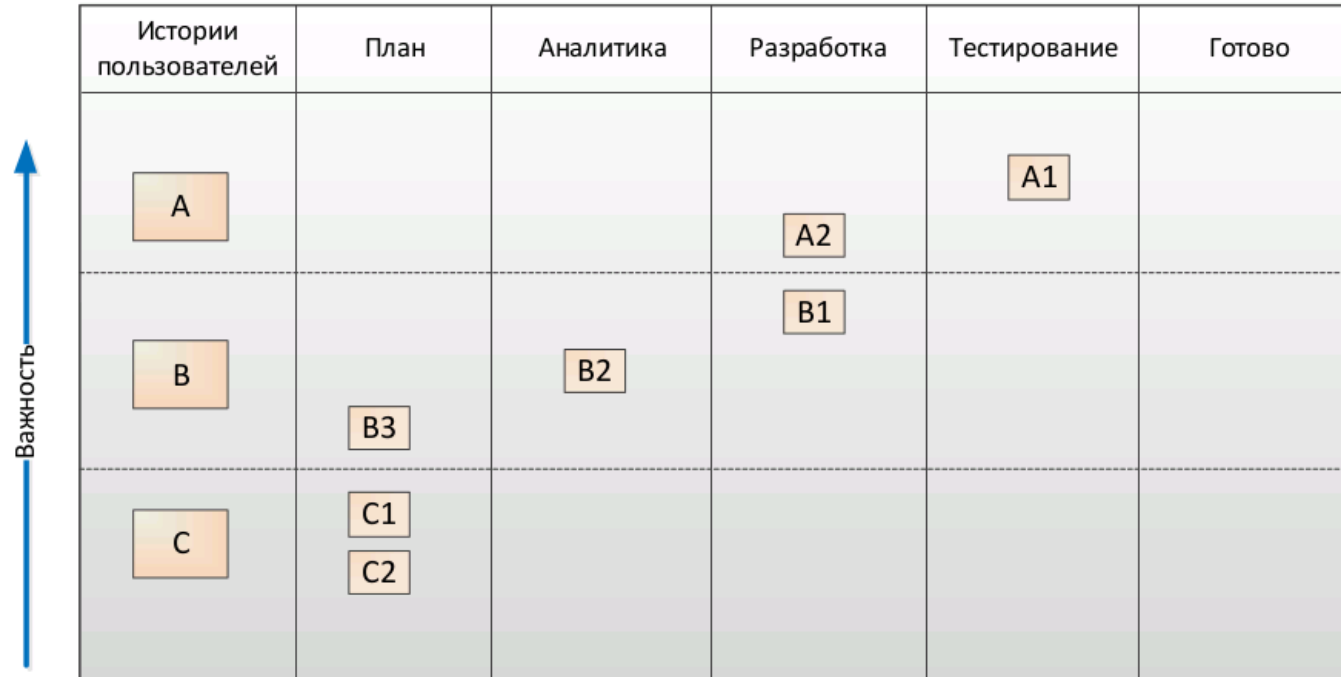
- Команда делает задачи по важности, начиная с самых верхних, доводя их до статуса «Готово»

Доска задач



- Вид диаграммы в конце спринта, если удалось реализовать все истории

Доска задач



- Достаточно большие истории в отдельном столбце
- Истории разбиваются на небольшие задачи

Использованные материалы

- «Гибкие методологии разработки»
Вольфсон Борис, версия 1.2
- www.agilemanifesto.org
agile-манифест, история его создания
- www.scrum.org
руководство и презентация по Скраму
- Историческая справка
<http://chernykh.net/content/view/993/1076/>
<http://streamwork.ru/upravlenie-proektami/348>
- www.slideshare.net
общедоступный хостинг презентаций
- www.wikipedia.org
свободная энциклопедия